

Laborator 5.

Proiectarea sistemului de operare în timp real MyOS. Contextul unui task. Comutarea task-urilor cu durata nelimitată

1. Obiectiv

În laboratoarele anterioare task-urile erau privite ca niște funcții cu durată finită în timp. Acestea erau apelate de către administratorul de task-uri. Într-o aplicație reală însă task-ul este o funcție cu durată nelimitată în timp (bucă infinită). În acest laborator vom vedea cum se poate realiza comutația între două funcții cu buclă infinită. De asemenea, la realizarea comutației este necesară salvarea setului de regiștrii și variabile care sunt folosite în interiorul task-ului și încărcarea lor ulterioară.

2. Breviar teoretic

Contextul unui task

Un task este de regula o funcție ce conține o buclă infinită. Un task este ca oricare funcție C dar care nu returnează nici o valoare. Tipul funcției trebuie să fie întotdeauna void. În MyOS un task este definit ca :

```
void taskN(void)
{
    for ( ;; ) {
        /* cod definit de utilizator */
    }
}
```

În timpul execuției un task utilizează regiștrii microcontrollerului și accesează memoriile RAM și ROM la fel ca orice alt program. Aceste resurse (regiștrii, stivă etc.) formează împreună contextul de execuție al task-ului. Un task este o porțiune de cod secvențială și care nu știe când va fi suspendat (oprit din execuție) sau reluat (îi se alocă timp de procesare suplimentar) de sistemul de operare. Să considerăm exemplul unui task care este suspendat imediat înainte de a executa o instrucțiune de adunare a valorilor conținute în doi regiștri.

Cât timp un task este suspendat, alte task-uri se vor executa și pot modifica valorile regiștrilor. Până la reluarea execuției task-ului suspendat acesta nu știe dacă regiștrii au fost modificați, astfel, dacă ar folosi valorile modificate, rezultatul operației de adunare ar fi unul eronat.

Pentru a preveni acest tip de eroare este esențial ca la reintrarea în execuție, un task să aibă un context identic cu cel anterior suspendării. Kernel-ul sistemului de operare este responsabil de salvarea contextului task-ului ce urmează a fi suspendat și de restaurarea contextului salvat atunci când task-ul este reluat. Procesul de salvare a contextului unui task ce urmează a fi suspendat și de restaurare a contextului unui task ce reintra în execuție se numește comutare de context (context switching).

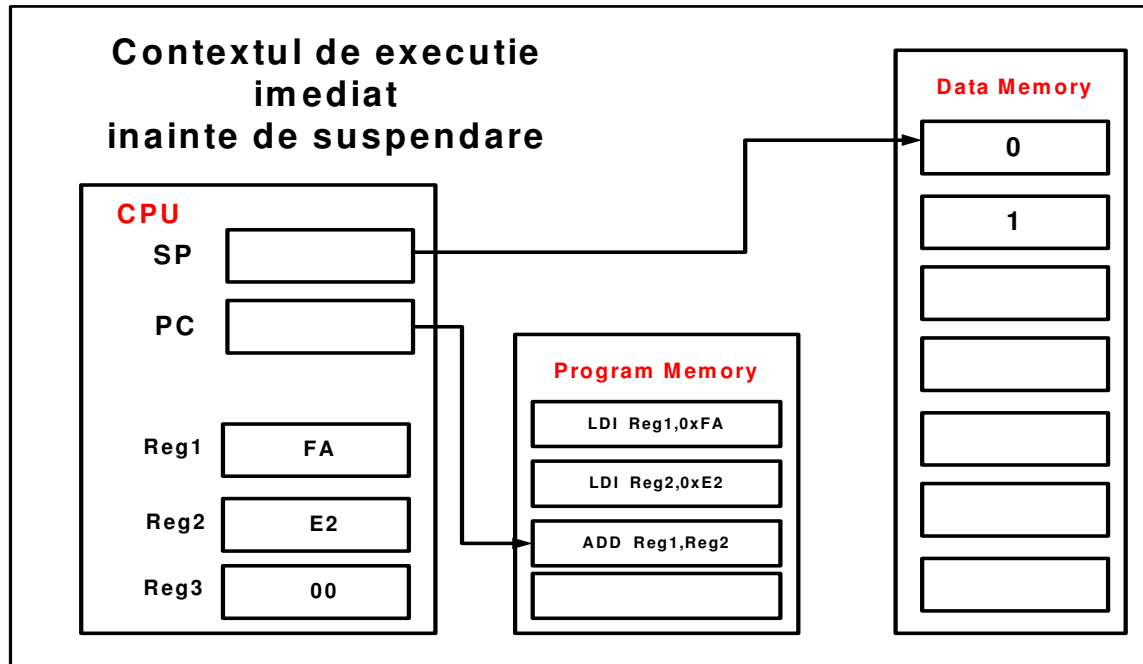


Fig. 1. Contextul de execuție

La un microcontroller AVR contextul este format din :

- 32 regiștri de uz general.
- Registrul de stare (Status Register). Valoarea registrului de stare afectează execuția instrucțiunilor, și trebuie să fie inclus în procesul de comutare de context.
- Registrul Program Counter. La reluare, un task trebuie să continue să fie executat de la instrucțiunea ce urma să se execute imediat după suspendarea task-ului.
- Cei doi regiștri ce formează pointerul către stivă (Stack Pointer).

Într-un program în C în regiștrii de uz generali sunt salvate variabilele locale, variabilele temporale folosite de compilator pentru diverse operații, parametrii funcțiilor și valorile returnate de acestea.

Alături de contextul AVR un task utilizează un set de variabile, locații din memoria RAM. În cazul în care aceste variabile sunt utilizate și de către alte task-uri este necesară o salvare a acestora înainte de a se efectua comutația.

Rezumând cele prezentate mai sus un task va trebui să aibă pe lângă corpul funcție care reprezintă ceea ce are de executat și următoarele :

- o zonă de memorie RAM în care se va afla stiva pentru task-ul respectiv, denumită stivă hard ;
- o zonă de memorie RAM în care se vor salva setul de regiștrii ;
- o zonă de memorie RAM în care se vor salva variabilele apelate și de ale task-uri ;
- o zonă de memorie RAM în care se vor salva cei doi regiștrii alocați pentru stivă SPH și SPL. Ultimele trei zone vor purta numele de stivă software.

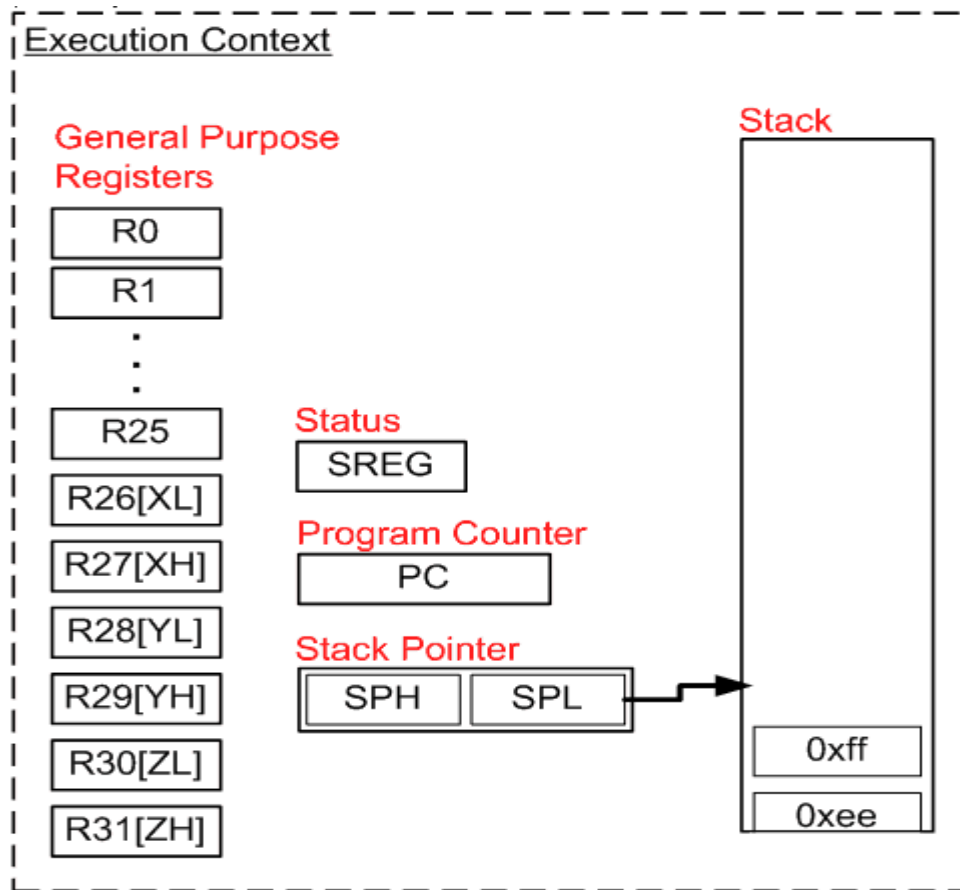


Fig.2. Contextul la ATMEGA128

Apelul unei subrutine la AVR

La apelul unei subrutine (funcții) sunt efectuate de către procesor următoarele operații:

- este salvat numărătorul de program (PC) care indică adresa curentă în memoria de program în stivă;
- se incrementează vârful stivei, registrul SP;
- se efectuează saltul la adresa subrutinei, prin încărcarea în PC a adresei subrutinei;
- se execută subrutina;
- se restaurează registrul PC cu valoarea salvată în stivă.

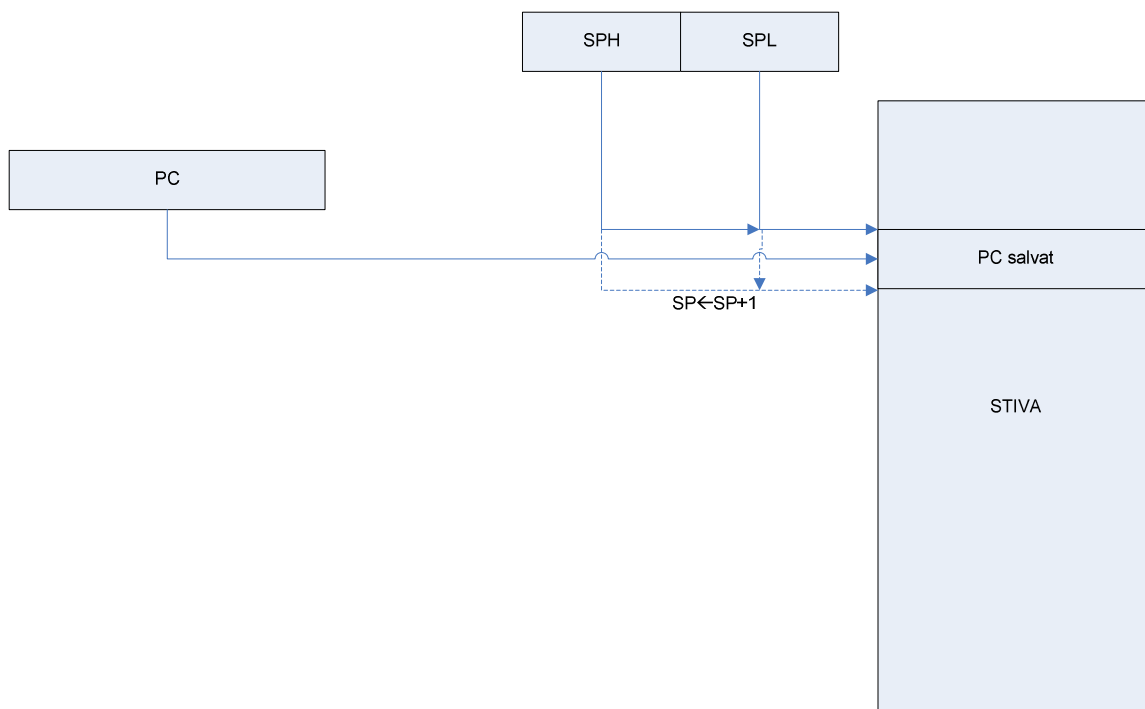


Fig.3. Salvarea în stivă a PC

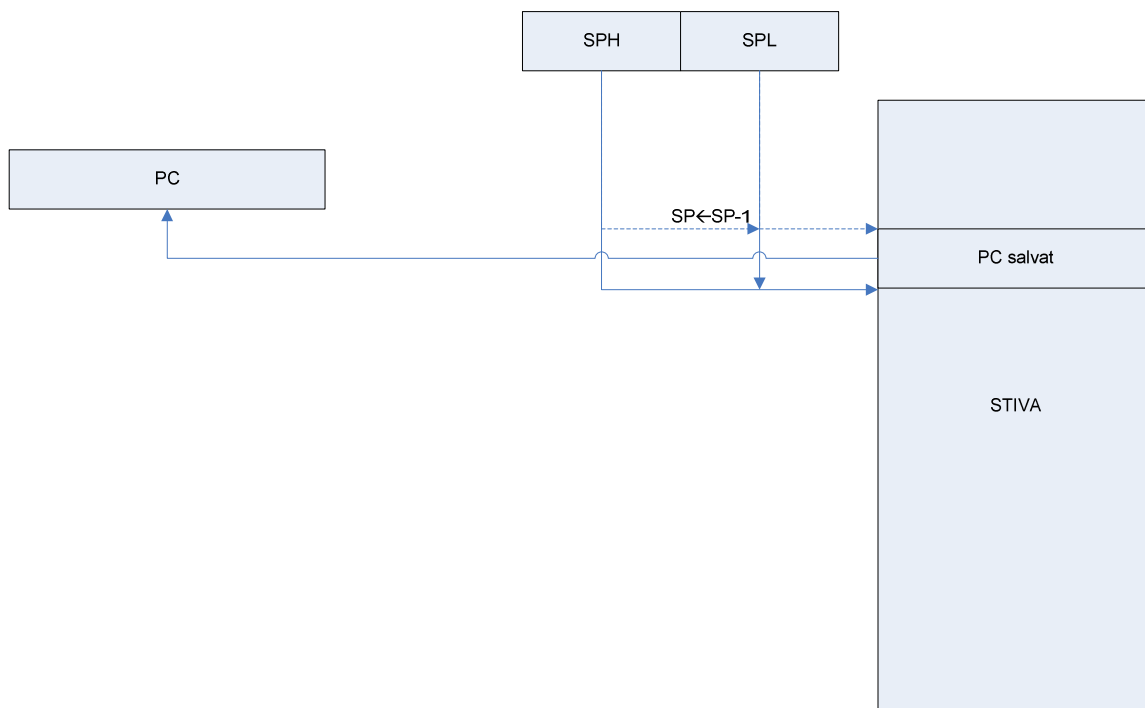


Fig. 4. Restaurarea din stivă a PC

Salvarea regiștrilor de uz general a lui ATMEGA

Microcontroler-ul ATMEGA128 are 32 de regiștrii de uz general notați cu R0 până la R31 (fig. 2). Salvarea regiștrilor în stivă se face prin folosirea unor instrucțiuni descrise în limbaj de asamblare. Pentru includerea în interiorul programului C a unor instrucțiuni de asamblare se folosește următoarea sintaxă:

```
asm(" instrucțiune asamblare ");
```

Pointeri la ATMEGA

Pentru adresarea indirectă a memoriei se folosesc pointeri X, Y sau Z. Pentru pointerul X sunt folosiți doi regiștrii de uz general, R26 (low) și R27 (high). În ei se încarcă adresa (pe 16 biți) a locației care apoi va fi scrisă.

InițIALIZAREA pointer X cu valoarea adresei la care se dorește să se efectueze salvarea

```
//initializarea adresei pointer
```

```
asm(" LDS R26,0x30");
```

```
asm(" LDS R27,0x00");
```

```
//scrierea în memorie a registrului R4
```

```
asm(" ST -X, R4");
```

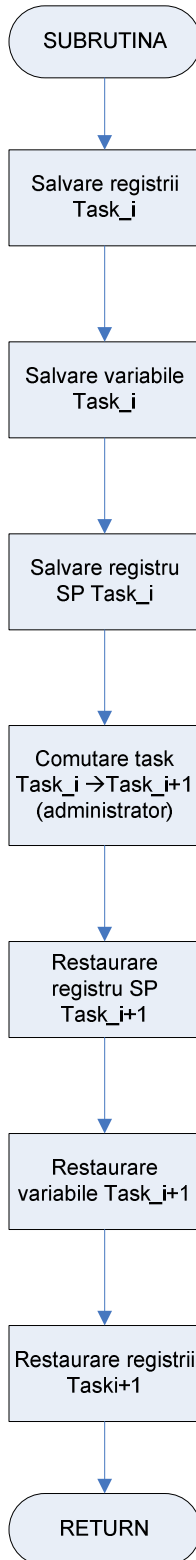
```
//citirea din memorie a registrului R4
```

```
asm(" LD R4,X+");
```

. Comutarea task-urilor

Se efectuează în interiorul subrutinei care tratează întreruperea de la timer. În figura de mai jos este prezentată secvența corectă pentru comutarea între task-uri. Operarea în această manieră este cea mai corectă și acoperă toate cazurile care pot apărea într-un sistem în timp real folosit în practică.

În problema de mai jos anumite aspecte legate de comutarea între task-uri și salvarea contextului au fost simplificate pentru a putea fi ușor înțelese. Astfel, conceptul de comutare între funcții s-a limitat la definirea unei variabile care va fi modificată de către administratorul de task-uri și care indică bucla (task-ul) curent care va fi executat precum și la salvarea unor variabile locale folosite de task.

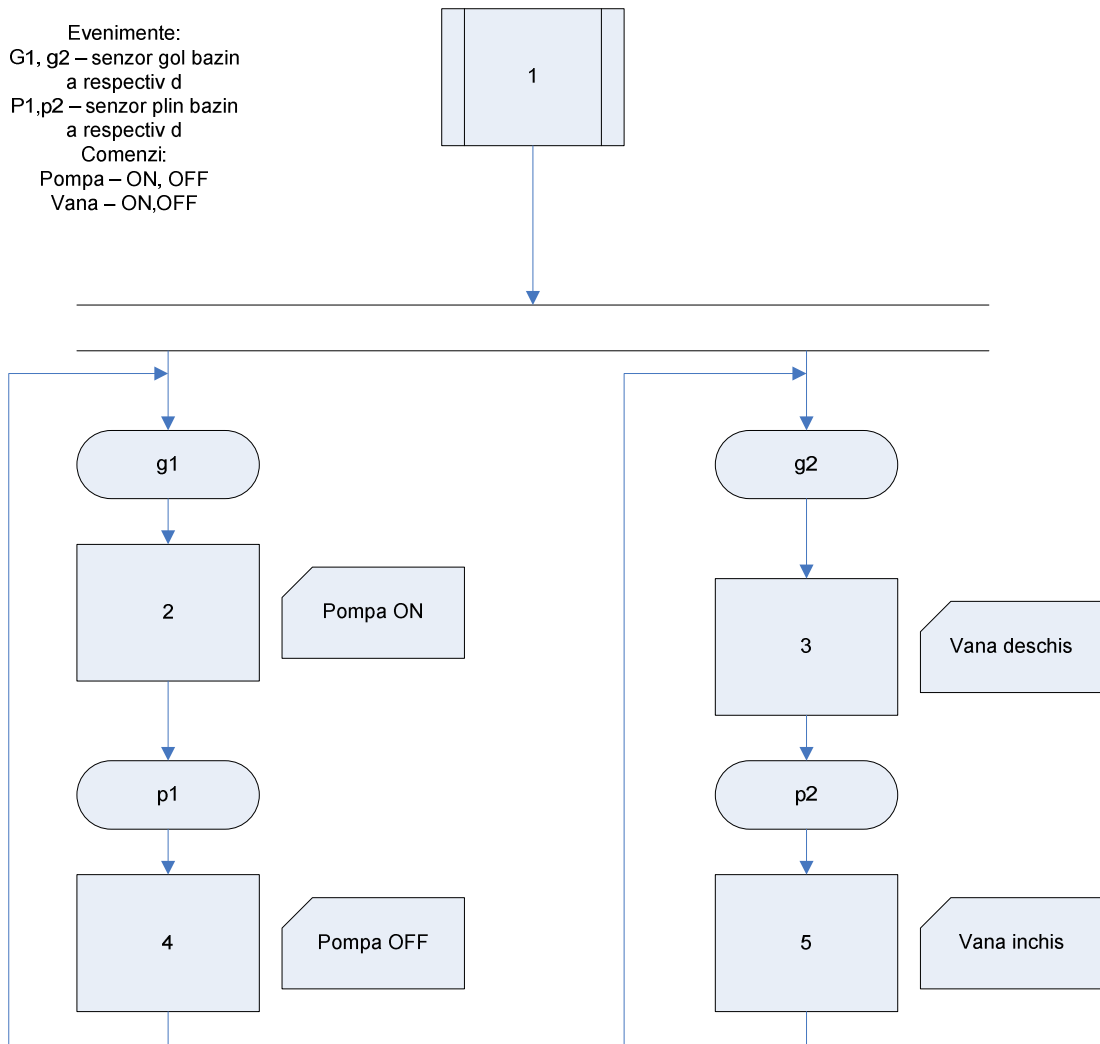


3. Desfășurarea lucrării

Pentru ilustrarea modului de lucru cu task-uri se formulează problema cu cele două bazine folosite pentru alimentarea cu apă reprezentată prin GRAFCET la curs:

Bazinul colector este înzestrat cu doi senzori: G1 care semnalizează lipsa apei și P1 care semnalizează faptul că este plin. Când acesta este gol trebuie pornită pompa de apă care îl va alimenta până când este plin.

Bazinul de distribuție este de asemenea înzestrat cu doi senzori: G2 care semnalizează faptul că este gol iar P2 faptul că este plin. Când acesta este gol trebuie deschisă vana care face legătura cu bazinul colector. Când acesta s-a umplut vana este închisă la loc. În plus față de aceste cerințe se definește timpul de răspuns maxim admisibil: 100 ms. Frecvența de lucru a sistemului este de 5MHz. În plus, se cere contorizarea numărului de acționări/opriri ale vanei și ale pompei.

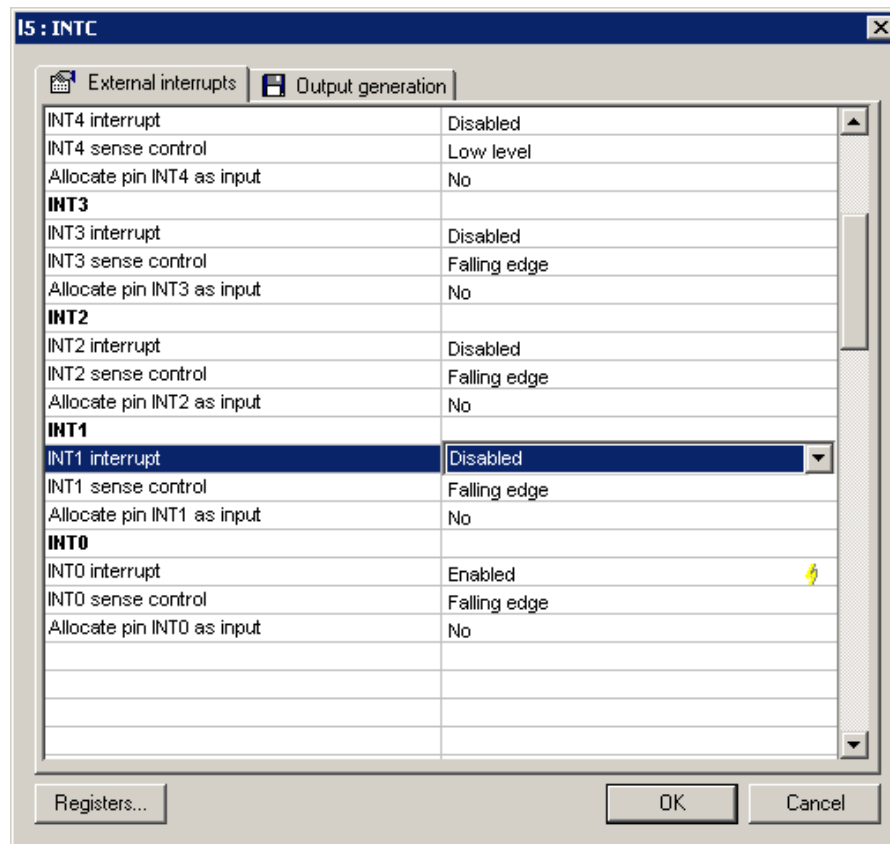


Atragem atenția că anumite etape prezentate aici au fost detaliate în laboratoarele anterioare!

1. CONFIGURAREA HARDWARE

Evenimentele provenite de la senzorii G1, G2, P1 și P2 vor fi simulate prin cereri de întreruperi externe. Se va folosi o singura intrerupere externă iar la apariția ei se va incrementa indicatorul de evenimente.

Prin urmare va fi configurată întreruperea externă INT0 pentru G1, P1, G2 respectiv P2. Se va utiliza MakeApp cu configurări pentru întreruperi externe (butonul INTC) pe front negativ:



Apoi la Output generation se va selecta generate all functions.

Cerința problemei impune un timp de răspuns de maxim admisibil (deadline time) de 100 ms. Se va alege un timp de comutare între task-uri de 10ms suficient pentru a îndeplini cerințele de timp ale problemei.

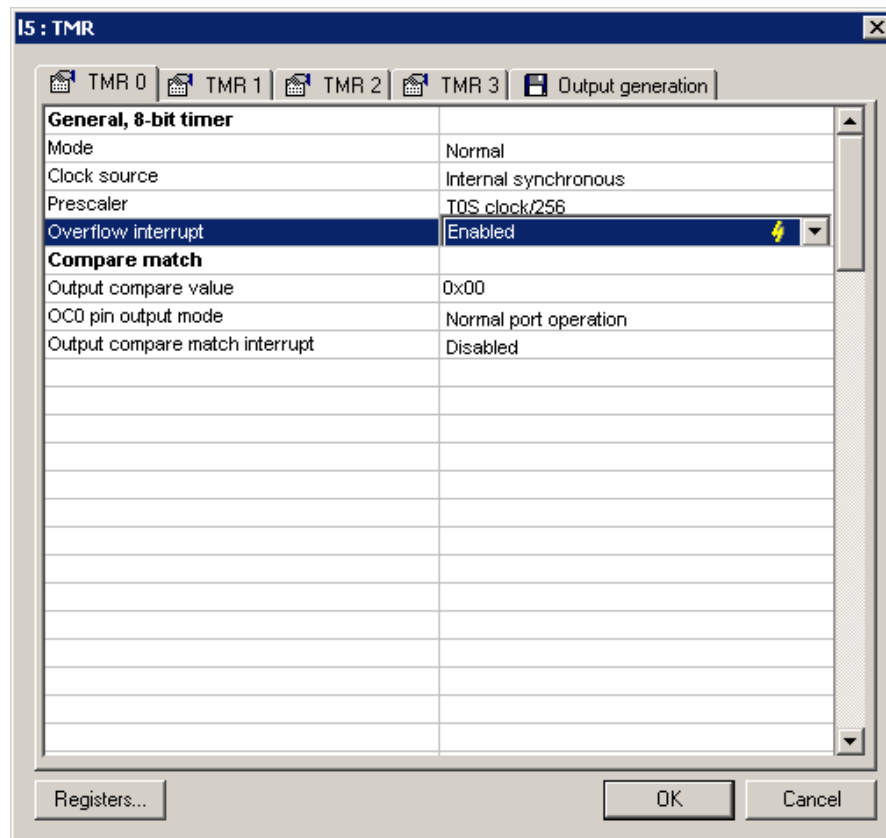
Dacă este posibil alegerea timpului de comutare între task-uri este de preferată să fie cu un ordin de mărime mai mic decât deadline time.

Prin urmare avem următoarele calcule pentru divizare:

Frecvența de pornire (microsistem) – 5MHz
Frecvența de comutare cerută de task – 10ms = 100Hz.

$5.000.000 / 256 = 19.531 / 100 \sim 195$ prescaler 256 divizare cu 195
Constanta de divizare va fi:
 $256 - 195 = 61$.

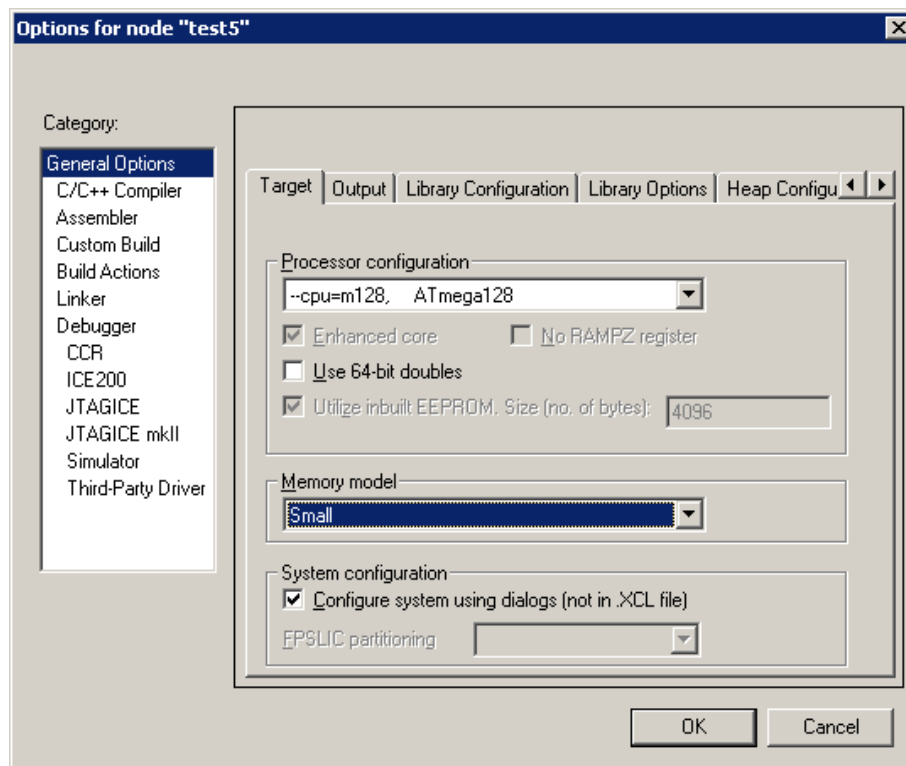
Deci canalul timer 0 (8 biți) se va programa cu prescaler 256, constată divizare 61, activare întrerupere. Programarea acestuia se face tot în MakeApp folosind instrumentul TMR:



2. CONFIGURAREA SOFTWARE

Reaminitim că librăriile generate trebuie prelucrate după ce au fost generate cu MakeApp (comentată incluziunea `#include "usercode.h"` și toate macrocomenzile care încep cu `ENTER_`, `INSIDE_` și `EXIT_`) și trebuie puse în același director cu aplicația principală.

La crearea proiectului în IAR nu se va uita să se seteze procesorul pe ATMEGA 128. După aceea se vor include în proiect fișierele librări create cu MakeApp: `ma_intc.c`, `ma_tmr.c`.



a. Definirea variabilelor

Se va crea un fișier cu numele `variabile.c`. În el se va scrie:

```
#include "variabile.h"
```

```
//variabile indicator evenimente  
unsigned char g1=0,p1=0,g2=0,p2=0;
```

```
//variabile memorie salvare context pentru task1 si pentru task2  
int context_task[NR_TASK][10];
```

```
//variabila task curent
```

```
unsigned char task_curent=0;
```

```
//variabila contor folosita de ambele task-uri  
int contor=0;
```

```
//contor evenimente  
unsigned char contor_ev=0;
```

Se va crea un fișier cu numele variabile.h. Acesta va conține declarațiile de variabile. Fișierul va fi inclus unde se va dori folosirea variabilelor:

```
//definire numar taskuri  
#define NR_TASK 2
```

```
//variabile indicator evenimente  
extern unsigned char g1,p1,g2,p2;  
extern unsigned char g1_s,p1_s,g2_s,p2_s;  
//variabile memorie pentru task-uri  
extern int context_task[NR_TASK][10];
```

```
//variabila task curent  
extern unsigned char task_curent;
```

```
extern int contor;  
extern unsigned char contor_ev;
```

b. Modificarea funcțiilor librării

În fișierul ma_intc unde se află funcțiile care lucrează cu întreruperile externe. La apariția unei întreruperi externe un eveniment se va produce. Acesta va fi, în ordinea în care au fost declarate, golire bazin 1 g1, umplere bazin 1 p1, golire bazin 2 g2 umplere bazin 2 p2. Subrutina se va suprascrie astfel:

```
#pragma vector=MA_INT0_vect  
__interrupt void MA_IntHandler_INT0_INTC( void )  
{  
  
switch(contor_ev)  
{  
case 0:  
g1=1;  
break;  
case 1:  
p1=1;  
break;  
case 2:
```

```

    g2=1;
    break;
case 3:
    p2=1;
    break;
}

contor_ev++;
if(contor_ev==4)
    contor_ev=0;

}

```

Fișierul `ma_tmr.c` se va suprascrie astfel:

```

#pragma vector=MA_TIMER0_OVF_vect
__interrupt void MA_IntHandler_OVF0_TMR( void )

{
    //salvare context task i
    context_task[task_curent][0]=contor;
    //comutare task
    task_curent++;
    if(task_curent==2)
        task_curent=0;
    //restaurare context task i+1
    contor=context_task[task_curent][0];
}

```

Se poate observa etapele principale ca salvare context, comutare context, restaurare context.

c. Declararea task-urilor și a programului principal:

Se va crea fișierul `main.c` și se va scrie următoarele:

```

#include <inavr.h>
#include "ma_tmr.h"
#include "ma_intc.h"
#include "variabile.h"
#include <stdio.h>

void task(void)
{

```

```

for(;;)
{
//task-ul 1
while(task_curent==0)
{
if(g1)
{
g1=0;
contor++;
printf("Porneste pompa %d\n",contor);
}
if(p1)
{
p1=0;
printf("Opreste pompa %d\n",contor);
}
}
}
//task-ul 2
while(task_curent==1)
{
if(g2)
{
g2=0;
contor++;
printf("Deschide vana %d\n",contor);
}
if(p2)
{
p2=0;
printf("Inchide vana %d\n",contor);
}
}
}
}

```

```

void main(void)
{
char i,j;
//initializare circuie hardware
MA_Init_INTC();
MA_Init_TMR();
MA_Start_TMR(0,1);
//initializare context
for(i=0;i<NR_TASK;i++)
for(j=0;j<10;j++)

```

```

    context_task[i][j]=0;
    __enable_interrupt();
    //pornire primul task!
    task_curent=0;
    task();
}

```

Task-urile au fost definite ca bucle while care rulează în funcție de variabila task_curent

Toate fișierele vor fi incluse în același director de lucru. În final ar trebui să avem:

Nume fișier	Specificări	Sursă
Ma_intc.c	Trebuie inclus în proiect	Generat cu makeapp – modificat
Ma_intc.h		Generat cu makeapp
Ma_tmr.c	Trebuie inclus în proiect	Generat cu makeapp – modificat
Ma_tmr.h		Generat cu makeapp
Ma_tgt.h	Utilizat de fișierele librărie	Generat cu makeapp
Ma_io.h	Utilizat de fișierele librărie	Generat cu makeapp
Main.c	Trebuie inclus în proiect	Creat
Main.h		Creat
Variabile.c	Trebuie inclus în proiect	Creat
Variabile.h		Creat

d. Rularea programului.

După ce s-a compilat, în simulator vor fi setate secvențele de întreruperi din meniu Simulator – Interrupts):

Întrerupere	Activation time	Repeat interval
INT0	100	100
TIMER0 OVF	10	10

Se va afișa fereastra Terminal I/O (din meniu View – Terminal I/O).

Se vor plasa break point la fiecare apel de funcție printf. Se va rula programul (Go). Se vor vizualiza (View – Watch) variabilele contor, contor_ev, task_crt. Se va urmări accesarea terminalului.

Întrebări. Aplicații

1. Contextul fiecărui task a fost creat de dimensiunea 10 deși practic o singură variabilă este practic salvată. Să se declare 3 variabile distincte care se vor actualiza la fiecare apel al task-ului astfel: prima se va incrementa din doi în doi, a doua va calcula suma între primele două iar a treia produsul. Să se realizeze salvarea acestora la apelul taskului.
2. Este posibil, fiind în primul task să putem apela variabile modificate de cel de-al doilea? Cum se poate realiza acest lucru?

3. Care este deficiența acestui sistem de administrare a task-urilor față de cel care utilizează registrul SP prezentat în breviarul teoretic?
4. Să se declare trei task-uri distincte care să poată fi administrate folosind această metodă. Un task scrie un octet într-o memorie inel de 100 octeți, al doilea citește un octet din memoria inel, îi aplică o secvență de cifrare xor cu valoarea 55h și depune rezultatul într-o a doua memorie inel de 100 octeți al treilea citește din această memorie și îl transmite serial.
5. Să se imagineze un administrator de taskuri cu durată nelimitată în timp care funcționează cu instrucțiuni de tipul goto.